



Laboratório de Conceitos | Desenvolvimento Web

Exemplos simples e didáticos para preparação da avaliação

 OneCompiler

 Ferramentas

 Variáveis

 Operadores

 Arrays

 CORS



Sobre este laboratório

Cada conceito é apresentado com explicação, exemplo prático e atividade.



Conceitos teóricos têm explicação detalhada em vez de código extenso.



Conceito 1: OneCompiler Teórico



O que você precisa saber:

- O OneCompiler é um **editor online**
- JavaScript é executado **no servidor** usando **Node.js**
- **Não** é executado no navegador do usuário
- `localStorage` , `document` e `window` **não funcionam**

1 O OneCompiler executa JavaScript em qual ambiente?

☐ a) No navegador

☒ b) No servidor com Node.js

☐ c) Em ambos

☐ d) Em máquina virtual

☒ Verificar

☒ Correta! OneCompiler executa no servidor com Node.js.



Conceito 2: Ferramentas

Teórico



O que você precisa saber:

- **VS Code** → Editor gratuito com extensões
- **GitHub Pages** → **GRATUITO** para sites públicos
- **Live Server** → Servidor local com recarga automática

2 Qual afirmação sobre ferramentas é **FALSA** ?

☐ a) VS Code é gratuito☒ b) GitHub Pages é pago para sites públicos☐ c) Live Server cria servidor local☐ d) VS Code suporta extensões☒ Verificar☒ Correta! GitHub Pages é GRATUITO para sites públicos.



Conceito 3: Variáveis - Reatribuição

JavaScript

✦ O que você precisa saber:

- **const** → NÃO pode mudar o valor ✗
- **let** → PODE mudar o valor ✓
- **var** → PODE mudar o valor ✓ (antigo, evitar)

reatribuicao.js JS

```
// const → NÃO pode mudar
const nome = "João";
console.log(nome);
// nome = "Maria"; // ERRO!

// let → PODE mudar
let idade = 25;
console.log(idade);
idade = 26;
console.log(idade);

// var → PODE mudar (antigo)
var cidade = "SP";
cidade = "RJ";
console.log(cidade);
```

Executar Localmente

Limpar

Clique em "Executar Localmente".

3 Qual palavra-chave NÃO permite reatribuição?

☐ a) var☐ b) let☒ c) const☐ d) Todas

☒ Verificar

☒ Correta! const não permite reatribuição.



Conceito 3B: Variáveis - Escopo

JavaScript

📌 O que você precisa saber:

- **let** e **const** → ficam presos ao **bloco { }**
- **var** → NÃO fica preso ao bloco, pode "escapar"

escopo.js JS

```
// Dentro do bloco { }  
if (true) {  
    let a = 10;  
    const b = 20;  
    var c = 30;  
    console.log("Dentro:", a, b, c);  
}  
  
// Fora do bloco  
console.log("Fora - var c =", c); // ✅ funciona  
// console.log(a); // ❌ ERRO! (descomente para testar)  
// console.log(b); // ❌ ERRO! (descomente para testar)
```

Executar Localmente

Limpar

Clique em "Executar Localmente".

4 Qual palavra-chave NÃO tem escopo de bloco?

☒ a) var☐ b) let☐ c) const☐ d) Nenhuma

Verificar

✓ Correta! var NÃO tem escopo de bloco.



Resumo: var, let e const

Declaração	Pode mudar?	Escopo	Boa prática
const	✗ Não	Bloco	✓ Use sempre
let	✓ Sim	Bloco	✓ Use quando precisar
var	✓ Sim	Função	✗ Evite



Use **const** por padrão, **let** quando precisar reatribuir, **NUNCA** use **var**.



Conceito 4: Operadores (== vs ===)

JavaScript



O que você precisa saber:

- == → compara **valor**, com conversão
- === → compara **valor E tipo**, sem conversão

operadores.js JS

```
const a = 10;  
const b = "10";  
  
console.log("10 == '10' →", a == b); // true  
console.log("10 === '10' →", a === b); // false  
  
console.log("✅ Sempre use ===");
```

Executar Localmente

Limpar

Clique em "Executar Localmente".

5 Qual é o resultado de `10 === "10"` ?

☐ a) true☒ b) false☐ c) undefined☐ d) null☒ Verificar

✅ Correta! === compara valor e tipo. 10 é number, `10` é string → false.



Conceito 5: Condicionais

JavaScript



O que você precisa saber: **if/else** e **else if** para múltiplas condições.

condicionais.js JS

```
const nota = 6;

let resultado;

if (nota >= 7) {
    resultado = "Aprovado";
} else if (nota >= 5) {
    resultado = "Recuperação";
} else {
    resultado = "Reprovado";
}

console.log("Nota:", nota);
console.log("Resultado:", resultado);
```

 Executar Localmente Limpar Clique em "Executar Localmente".

6 Com `nota = 4` , qual é a saída?

☐ a) Aprovado☐ b) Recuperação☒ c) Reprovado☐ d) Nenhuma☒ Verificar

✓ Correta! nota=4 < 5 → Reprovado.



Conceito 6: Arrays

JavaScript

📌 Métodos principais:

- **push()** → FINAL
- **pop()** → remove FINAL
- **shift()** → remove INÍCIO
- **unshift()** → INÍCIO
- **splice()** → qualquer posição

arrays.js JS

```
}

frutas.push("Uva");
mostrar("push('Uva')");

frutas.shift();
mostrar("shift()");

frutas.pop();
mostrar("pop()");

frutas.unshift("Morango");
mostrar("unshift('Morango')");

frutas.splice(1, 1, "Kiwi");
mostrar("splice(1, 1, 'Kiwi')");
```

[▶ Executar Localmente](#)[🧹 Limpar](#)

i Clique em "Executar Localmente".

7 Qual método adiciona no **INÍCIO** ?

☐ a) push()

☐ b) pop()

☒ c) unshift()

☐ d) shift()

☒ Verificar

☒ Correta! unshift() adiciona no INÍCIO.

8 O que faz `splice(2, 1, "Novo")` ?

☐ a) Adiciona no final

☒ b) Remove 1 na posição 2 e insere "Novo"

☐ c) Remove 2 a partir da posição 1

☐ d) Adiciona no início

☒ Verificar

☒ Correta! splice(2, 1, `Novo`) remove 1 na posição 2 e insere `Novo`.



Conceito 7: CORS

Teórico

📌 O que você precisa saber:

- **CORS** = política de segurança do navegador
- Bloqueia requisições entre **origens diferentes**
- Arquivos JSON locais (file://) são **bloqueados**
- **Solução:** servidor local (Live Server, http-server)



Por que JSON local não funciona? O navegador bloqueia por segurança. Use Live Server.

9 Por que JSON local (file://) é bloqueado?

☐ a) Navegador não suporta JSON

☒ b) Por segurança (CORS)

☐ c) Arquivo corrompido

☐ d) JSON é obsoleto

☒ Verificar

☒ Correta! CORS bloqueia por segurança. Use servidor local.



Resumo para a Avaliação



OneCompiler

Executa JS no servidor (Node.js).



Ferramentas

VS Code (grátis), GitHub Pages (grátis).



Variáveis

const (não reatribui), let (reatribui), var (evitar).



Operadores

== (valor), === (valor e tipo).



Arrays

push/pop (final), shift/unshift (início), splice (qualquer).



CORS

Bloqueia origens diferentes. Use servidor local.



Boa prova! Revise cada conceito e pratique com as atividades.

